

Object Oriented Programming & Design Principles

60 minutos · 5 bloques · Teoría profunda + entrevistas técnicas

OOP

SOLID

Composition

IoC / DI

Encapsulación

Abstracción

Herencia

Polimorfismo

SOLID · Composición · IoC

Agenda de la clase

0 – 10 MIN

01

¿Por qué existe OOP?

La Crisis del Software de 1968, problemas del código procedural y los 4 problemas que OOP viene a resolver

10 – 30 MIN

02

Los 4 Pilares de OOP en Java

Encapsulación → Abstracción → Herencia → Polimorfismo — en orden pedagógico, cada uno desde el problema que resuelve

30 – 48 MIN

03

Principios SOLID

Los 5 principios explicados desde el problema real que resuelven, con ejemplos de industria y preguntas de entrevista

48 – 55 MIN

04

Composición sobre Herencia

Cuándo la herencia falla, qué es la composición y cómo el patrón Strategy lo resuelve

¿Por qué existe OOP?

La Crisis del Software · El paradigma procedural · La respuesta

La Crisis del Software — 1968

⚠ El mundo antes de OOP

Los programas crecían a **decenas de miles de líneas** de código procedural sin estructura clara.

- | Variables **globales** accesibles desde cualquier lugar del programa
- | Estado del sistema **impredecible** — cualquier función podía mutar cualquier dato
- | Cambiar una función podía **romper diez** partes no relacionadas
- | Código imposible de **probar** de forma aislada



The cost of maintaining software is now many times the cost of building it.

— NATO Software Engineering Conference, 1968

Este evento acuñó el término **Software Crisis**

La industria necesitaba una nueva forma de organizar el código. La respuesta fue **modelar el software como el mundo real: objetos con estado y comportamiento propio.**

Los 4 problemas que OOP resuelve



Acoplamiento descontrolado



Encapsulación — el estado solo cambia a través de los métodos del objeto



Duplicación de lógica



Herencia y Polimorfismo — define el comportamiento común una vez y especializa solo lo que cambia



Rigidez al cambio



Abstracción + SOLID — extiende el sistema sin modificar código que ya funciona



Dominios complejos



Objetos y contratos — modela entidades del negocio de forma comunicable entre todo el equipo

OOP EN UNA LÍNEA

Organizar el software como **objetos** que combinan **estado** (datos) y **comportamiento** (métodos), que se comunican entre sí a través de **mensajes bien definidos**.

Los 4 Pilares de *OOP*

Encapsulación · Abstracción · Herencia · Polimorfismo

El orden importa — ¿por qué este orden?

1

Encapsulación

Base de todo: aprendemos a **ocultar estado** y exponer solo lo necesario. Es independiente, no necesita los otros pilares.

2

Abstracción

Introducimos **clases abstractas e interfaces** antes de herencia, porque la herencia las usa. Sin entender abstracción, la herencia es confusa.

3

Herencia

Ahora podemos hablar de **extender clases abstractas e implementar interfaces** con todo el contexto necesario.

4

Polimorfismo

El pilar más poderoso. Requiere entender **interfaces** (abstracción) y **herencia**. Lo natural es dejarlo para el final.

Pilar 1 — Encapsulación

PILAR 01

Agrupar **estado + comportamiento** en una unidad, y **proteger el estado interno** del acceso exterior no autorizado. El objeto es el *guardián de sus propias invariantes*.

✗ Encapsulación falsa

- Campos `private` con getter y *setter* para cada uno
- El objeto no valida nada — acepta cualquier valor
- El estado puede quedar en un valor **incoherente**
- Es una **clase anémica**: datos sin comportamiento real

Exposición de datos disfrazada

✓ Encapsulación real

- Campos siempre `private`, sin setters innecesarios
- Solo se exponen **operaciones de negocio** (`deposit`, `withdraw`)
- El objeto valida **antes** de modificar su estado
- Las colecciones internas se retornan como **copias defensivas**

Invariantes siempre garantizadas



Analogía — el cajero automático: No tienes acceso directo al efectivo. Interactúas a través de operaciones (*Retirar, Depositar, Consultar saldo*). El cajero valida antes de ejecutar. Así funciona la encapsulación real.

Encapsulación — Modificadores de Acceso en Java

Java provee 4 niveles de visibilidad. La regla de oro: **siempre empieza con el más restrictivo** y abre solo cuando es necesario.

MODIFICADOR	MISMA CLASE	MISMO PAQUETE	SUBCLASES	TODO EL MUNDO	CUÁNDO USARLO
<code>private</code>	✓	✗	✗	✗	Siempre para campos de instancia. Por defecto.
<code>package-private</code>	✓	✓	✗	✗	Colaboraciones internas dentro del mismo módulo.
<code>protected</code>	✓	✓	✓	✗	Solo cuando diseñas jerarquías de herencia deliberadas.
<code>public</code>	✓	✓	✓	✓	Solo para métodos que forman el contrato público del objeto.



Copia defensiva — cuando retornas una colección que es parte del estado interno, nunca devuelvas la referencia directa. El caller podría mutar tu estado sin pasar por tus métodos. Usa `Collections.unmodifiableList()` o `List.copyOf()`.

Encapsulación — Preguntas de Entrevista EPAM

¿Cuál es la diferencia entre encapsulación e information hiding? ¿Son lo mismo?

Son conceptos relacionados pero distintos. **Encapsulación** es el mecanismo: agrupar estado y comportamiento en una unidad. **Information hiding** (Parnas, 1972) es el principio de diseño: los detalles de implementación deben ser inaccesibles. La encapsulación es el mecanismo que *permite* aplicar information hiding. Un objeto puede estar "encapsulado" (campos privados) pero aun así exponer su implementación con demasiados getters/setters. Information hiding va más allá: la API no debe revelar *cómo* funciona internamente.

¿Tener campos `private` con un getter y setter para cada uno es buena encapsulación?

No. Es una **clase anémica** disfrazada de encapsulación. Si tienes `setBalance(double b)`, cualquier código externo puede poner el balance en `-99999` sin ninguna validación. La encapsulación real expone **operaciones de negocio** (`deposit()`, `withdraw()`) que internamente validan las reglas antes de modificar el estado. La pregunta correcta no es "¿tengo los campos privados?" sino "¿puede el estado quedar en un valor incoherente desde afuera?"

Pilar 2 — Abstracción

PILAR 02

Identificar los **conceptos esenciales** de un dominio y representarlos sin revelar los detalles de implementación.
Separar el "qué" del "cómo".

¿Por qué surge la abstracción?

Cuando tienes múltiples implementaciones de un mismo concepto (múltiples tipos de pago, múltiples tipos de reportes, múltiples repositorios), necesitas una forma de decir *"todo esto comparte este contrato"* sin repetir código. La abstracción es ese mecanismo.

Analogía: Un control remoto es una abstracción. No sabes si hay un televisor Samsung o LG detrás. Solo sabes que tiene botones "subir volumen", "cambiar canal", "apagar". El contrato es el mismo; la implementación varía.

Java tiene dos mecanismos de abstracción

Clase Abstracta

ABSTRACCIÓN PARCIAL

Puede tener **estado**, constructores, métodos concretos y métodos abstractos. Define un contrato + comportamiento base.

Interfaz

ABSTRACCIÓN PURA

Solo define **comportamiento**. Sin estado (solo constantes). Permite a una clase cumplir *múltiples contratos* simultáneamente.

Abstracción — Clase Abstracta vs Interfaz

	Clase Abstracta	Interfaz
Estado (campos de instancia)	✓ Sí — puede tener campos	✗ No — solo constantes <code>static final</code>
Constructores	✓ Sí	✗ No
Métodos con implementación	✓ Sí — métodos concretos	✓ Sí — <code>default</code> (Java 8+)
Métodos sin implementación	✓ Sí — métodos <code>abstract</code>	✓ Sí — métodos abstractos
Herencia múltiple	✗ Solo una clase padre	✓ Una clase puede implementar muchas
Relación que expresa	IS-A — "es un tipo de"	CAN-DO — "puede hacer esto"
Úsala cuando...	Necesitas estado compartido + Template Method Pattern	Defines un contrato puro de comportamiento sin estado

Abstracción — Programar hacia Contratos

"Program to interfaces, not implementations"

— GoF, Design Patterns (1994)

✗ Programar a implementaciones

- Tu clase crea directamente `new MySQLUserRepository()`
- No puedes cambiar de MySQL a PostgreSQL sin tocar esa clase
- No puedes probar sin una base de datos real
- Cada cambio de infraestructura rompe la lógica de negocio

✓ Programar a contratos

- Tu clase depende de `UserRepository` (la interfaz)
- Puedes inyectar `JpaUserRepository`, `MongoUserRepository`, o un stub de prueba
- La lógica de negocio no sabe ni le importa cómo se persiste
- El sistema es testeable, intercambiable y extensible



Regla práctica: declara tus variables, parámetros y tipos de retorno con el tipo más abstracto posible que tenga sentido en ese contexto. En lugar de `ArrayList<User>`, usa `List<User>`. En lugar de `JpaUserRepository`, usa `UserRepository`. Esto es lo que hace posible la Inyección de Dependencias, que veremos al final de la clase.

Abstracción — Preguntas de Entrevista EPAM

¿Cuándo usarías una clase abstracta y cuándo una interfaz? ¿Pueden coexistir?

Usa interfaz cuando defines un contrato de comportamiento que clases no relacionadas puedan implementar (`Comparable` , `Runnable` , `Serializable`). **Usa clase abstracta** cuando tienes una jerarquía con estado y comportamiento compartido que debe heredarse (el Template Method Pattern es el ejemplo canónico). Pueden coexistir perfectamente: una clase abstracta puede implementar interfaces, y una clase concreta puede extender una clase abstracta e implementar varias interfaces.

¿Qué es el Template Method Pattern y cómo se relaciona con la abstracción?

Es un patrón donde la clase abstracta define el **esqueleto de un algoritmo** en un método `final` , y delega los pasos variables a métodos `abstract` que las subclasses implementan. La clase abstracta provee la estructura; las subclasses proveen los detalles. Ejemplo real: un generador de reportes define el flujo (validar datos → generar encabezado → generar cuerpo → ensamblar → retornar), donde "generar encabezado" y "generar cuerpo" son abstractos y los implementan `PdfReportGenerator` , `ExcelReportGenerator` , etc.

Pilar 3 — Herencia

PILAR 03

Mecanismo que permite crear una nueva clase basada en otra existente, **reutilizando y especializando** su comportamiento. Establece una relación **IS-A** entre subclase y superclase.

Lo que hereda una subclase

- Todos los campos `protected` y `public` de la superclase
- Todos los métodos no-privados — puede usarlos tal cual o sobrescribirlos con `@Override`
- Debe llamar al constructor del padre con `super()` — explícito o implícito

Sealed Classes — Java 17+

Nuevo

- Con `sealed class permits A, B, C` controlas exactamente qué clases pueden extender la tuya
- El compilador verifica exhaustividad — ideal para modelar tipos algebraicos
- Recuperas control sobre tu jerarquía de herencia sin prohibirla completamente con `final`

⚠ Limitaciones críticas que debes conocer

Herencia simple: una clase solo puede extender una sola superclase. Diseño deliberado de Java para evitar el "diamante de la muerte" de C++.

Fragile Base Class Problem: cambiar la superclase puede romper subclases silenciosamente. El compilador no avisa. Los bugs aparecen en runtime.

Acoplamiento máximo: la herencia crea el acoplamiento más fuerte que existe en OOP. Úsala con criterio — no para reutilizar código, sino para reutilizar *comportamiento* *polimórfico*.

Herencia — Preguntas de Entrevista EPAM

¿Cuál es el "fragile base class problem" y cómo lo mitigas?

Cuando modificas el comportamiento interno de una superclase, las subclasses que dependen de ese comportamiento pueden romperse de formas inesperadas — y el compilador no lo detecta. Por ejemplo, si la superclase cambia el orden en que llama a métodos internos que las subclasses sobrescriben, el resultado puede ser incorrecto sin ningún error de compilación. Se mitiga con: (1) diseñar para herencia o prohibirla con `final`; (2) documentar explícitamente qué métodos son seguros de sobrescribir; (3) preferir composición sobre herencia cuando la jerarquía no es estable; (4) en Java 17+, usar `sealed` para controlar quién puede extender.

¿Para qué sirven las `sealed classes` de Java 17+ y cuándo las usarías?

Las sealed classes restringen explícitamente qué clases pueden extender o implementar una clase/interfaz, usando la cláusula `permits`. Son ideales para modelar **tipos algebraicos de datos (ADTs)**: cuando el número de variantes de un tipo es finito y conocido en tiempo de diseño (por ejemplo: `Shape` que solo puede ser `Circle`, `Rectangle` o `Triangle`). El compilador puede entonces verificar exhaustividad en un `switch` expression con pattern matching — si no cubres todas las variantes, hay un error de compilación. Perfectas en dominio rico con DDD.

Pilar 4 — Polimorfismo

PILAR 04

Capacidad de que objetos de **diferentes tipos** sean tratados a través de una **interfaz común**, y que el comportamiento concreto se determine según el tipo *real* del objeto.

Polimorfismo Estático

OVERLOADING · TIEMPO DE COMPILACIÓN

¿QUÉ ES? Múltiples métodos con el mismo nombre y diferente firma en la misma clase

¿CUÁNDO? Java decide cuál llamar en tiempo de **compilación** según el tipo estático del argumento

BINDING **Early binding** — el método a llamar se fija antes de ejecutar

EJEMPLO `String.valueOf(int)` , `String.valueOf(char)` , `String.valueOf(boolean)`

Polimorfismo Dinámico

OVERRIDING · TIEMPO DE EJECUCIÓN

¿QUÉ ES? Una subclase provee su propia implementación de un método heredado con `@Override`

¿CUÁNDO? La JVM decide cuál implementación llamar en **runtime** según el tipo real del objeto en memoria

BINDING **Late binding** — el método se resuelve en tiempo de ejecución

PODER Una `List<PaymentProcessor>` puede tener Stripe, PayPal y Crypto mezclados — el código que la usa no cambia al agregar uno nuevo

Polimorfismo — La trampa de overriding vs. method hiding

El polimorfismo dinámico es el corazón de la extensibilidad real. Es lo que hace posible el principio **Open/Closed**: código que acepta una abstracción funciona sin cambios cuando añades nuevas implementaciones.

⚠️ Pregunta trampa frecuente en EPAM

"¿Se puede hacer overriding de un método `static`?"

No. Los métodos `static` pertenecen a la *clase*, no al *objeto*. Se resuelven en tiempo de compilación (early binding). Lo que parece overriding de un método estático en una subclase se llama **method hiding**, que es fundamentalmente diferente: depende del tipo *declarado* de la variable, no del tipo real del objeto.

🚫 ¿Qué métodos NO se pueden sobrescribir?

- `static` — pertenecen a la clase, no al objeto (method hiding)
- `final` — el compilador lo prohíbe explícitamente
- `private` — no son visibles en la subclase
- Constructores — no se heredan, por ende no se sobrescriben

El verdadero poder: si tienes `List<Shape>` con `Circle`, `Rectangle` y `Triangle` mezclados, puedes llamar `shape.calculateArea()` en cada uno y la JVM despacha automáticamente a la implementación correcta. Agregar `Pentagon` no cambia *ni una sola línea* del código que procesa la lista.

Principios **SOLID**

Robert C. Martin · 5 principios · El estándar de diseño en la industria Java

SOLID — ¿Qué son y por qué importan?

SOLID no son reglas matemáticas — son **guías de razonamiento** que te ayudan a hacer las preguntas correctas durante el diseño. No se aprenden para aplicarlos mecánicamente; se aprenden para *detectar cuándo el código va a ser un problema*.

S**Single Responsibility**

Una clase, una razón para cambiar

O**Open / Closed**

Abierto a extensión — cerrado a modificación

L**Liskov Substitution**

Una subclase debe poder sustituir a su superclase sin romper el programa

I**Interface Segregation**

Los clientes no deben depender de interfaces que no usan

D**Dependency Inversion**

Depende de abstracciones, no de implementaciones concretas

S — Single Responsibility Principle

S

Single Responsibility Principle

"Una clase debe tener una, y solo una, razón para cambiar."

● El Problema — La clase "Dios"

Una clase que maneja validación de negocio, consultas a base de datos, envío de emails y generación de reportes tiene **cuatro razones para cambiar**. Cambiar el proveedor de email obliga a tocar la misma clase que contiene la lógica de negocio más crítica. ¿El resultado? Nadie quiere tocar ese archivo. Cada cambio se convierte en una ruleta rusa.

● La Solución

Separar en clases con responsabilidades únicas: **OrderProcessor** (solo orquesta el flujo), **OrderRepository** (solo persiste), **NotificationService** (solo notifica), **ReportService** (solo genera reportes). Cambiar el proveedor de email solo toca `NotificationService`. La lógica de negocio no se toca.

Pregunta para detectar violaciones: "¿Quién o qué cambiaría si esta clase necesita cambiar?" Si la respuesta tiene más de un actor (el equipo de infraestructura, el equipo de negocio, el equipo de front-end), la clase viola SRP.

O — Open / Closed Principle



Open / Closed Principle

"Las entidades de software deben estar abiertas para extensión, pero cerradas para modificación."

● El Problema

Tienes un `DiscountCalculator` con un `switch` que evalúa el tipo de descuento. Cada vez que el negocio decide un nuevo tipo de descuento (estudiante, lealtad, temporada navideña) tienes que **modificar** ese `switch`. Cada modificación es una oportunidad de introducir un bug en descuentos que ya funcionaban correctamente.

● La Solución

Defines una interfaz `DiscountPolicy` con los métodos `isApplicable()` y `calculateDiscount()`. Cada tipo de descuento es una nueva **clase separada** que implementa esa interfaz. El motor que aplica descuentos recibe una lista de políticas y las evalúa. Agregar `ChristmasDiscountPolicy` no toca ni una línea del motor.

MECANISMO

OCP se implementa principalmente con **polimorfismo dinámico**: defines un contrato abstracto y cada nueva variante es una nueva implementación. También se logra con el **Strategy Pattern** y el **Decorator Pattern**.

L — Liskov Substitution Principle



Liskov Substitution Principle — Barbara Liskov, 1987 · Turing Award 2008

"Si S es un subtipo de T , los objetos de tipo T pueden ser reemplazados por objetos de tipo S sin alterar la correctitud del programa."

✗ Violación clásica — El Pingüino

Penguin extiende Bird. Bird tiene el método `fly()`.
Penguin sobrescribe `fly()` lanzando `UnsupportedOperationException`. El código que recibe una `List<Bird>` y llama `bird.fly()` **explota en runtime** cuando hay un pingüino — aunque compiló perfectamente. LSP roto.

✓ Diseño correcto

Separar las capacidades en interfaces: `FlyingBehavior` y `SwimmingBehavior`. El águila implementa solo `FlyingBehavior`. El pingüino implementa solo `SwimmingBehavior`. El pato implementa ambas. Cada objeto solo promete lo que puede cumplir.

La lección clave: la relación IS-A de OOP no es la misma que la relación matemática de subconjunto. Matemáticamente un cuadrado ES un rectángulo, pero si `Square` extiende `Rectangle` y el código puede cambiar ancho y alto independientemente, `Square` rompe ese contrato. **LSP es sobre comportamiento observable, no sobre propiedades matemáticas.**

I — Interface Segregation Principle



Interface Segregation Principle

"Los clientes no deben ser forzados a depender de interfaces que no usan."

❌ Interfaz gorda — el problema

Una interfaz `MultifunctionDevice` declara: `print()`, `scan()`, `fax()`, `staple()`, `copy()`. Una impresora básica que solo imprime está **forzada a implementar** los cuatro métodos restantes, probablemente lanzando `UnsupportedOperationException` en todos. Esto viola también LSP. El que usa la interfaz no sabe cuáles métodos funcionan realmente.

✅ Interfaces segregadas — la solución

Cuatro interfaces pequeñas: `Printable`, `Scannable`, `Faxable`, `Stapleable`. La impresora básica implementa solo `Printable`. El dispositivo avanzado implementa las cuatro. El código que solo necesita imprimir depende solo de `Printable`: no sabe ni le importa si el dispositivo también puede escanear.

ISP en Java estándar: las interfaces del JDK son el mejor ejemplo. `Runnable` (1 método), `Callable` (1 método), `Comparable` (1 método), `Supplier`, `Consumer`, `Function`, `Predicate`. Cada una hace exactamente una cosa. No hay una interfaz `EverythingYouNeed`. Esto además es lo que permite usarlas como lambdas en Java 8+.

D — Dependency Inversion Principle



Dependency Inversion Principle

"Los módulos de alto nivel no deben depender de los de bajo nivel. Ambos deben depender de abstracciones."

✗ Sin DIP — dependencia directa

AuthService (Alto nivel)

↓ depende de

MySQLUserRepository (Bajo nivel)

Problema: cambiar MySQL por PostgreSQL obliga a modificar
AuthService — tu lógica de negocio más crítica

✓ Con DIP — ambos dependen de abstracción

AuthService (Alto nivel)

↓↑ ambos dependen de

«interface» UserRepository

↑ implementa

JpaUserRepository (Bajo nivel)

El beneficio concreto: puedes crear `InMemoryUserRepository` solo para tests, sin base de datos, sin Docker, sin configuración.
`AuthService` no se entera — recibe la interfaz, no la implementación. Esto es lo que hace el código **testable** e **intercambiable**.

Composición sobre Herencia

"Favor composition over inheritance" — GoF, Design Patterns, 1994

Los 4 problemas reales de la Herencia

01

Fragile Base Class

Modificar la superclase puede romper subclases silenciosamente. El compilador no avisa. Los bugs aparecen en runtime en producción.

02

Herencia Simple

Una clase solo puede extender una superclase. Un `FlyingFish` que vuela Y nada no puede heredar de `Bird` y `Fish` simultáneamente.

03

IS-A Falso

Muchos usan herencia solo para reutilizar código sin que la relación IS-A sea genuina. Esto viola LSP y produce jerarquías incoherentes.

04

Explosión Combinatorial

Con 3 comportamientos independientes (Volar, Nadar, Correr) necesitarías 7 subclases para cubrir todas las combinaciones. Con 4: 15 subclases.

La solución — Composición

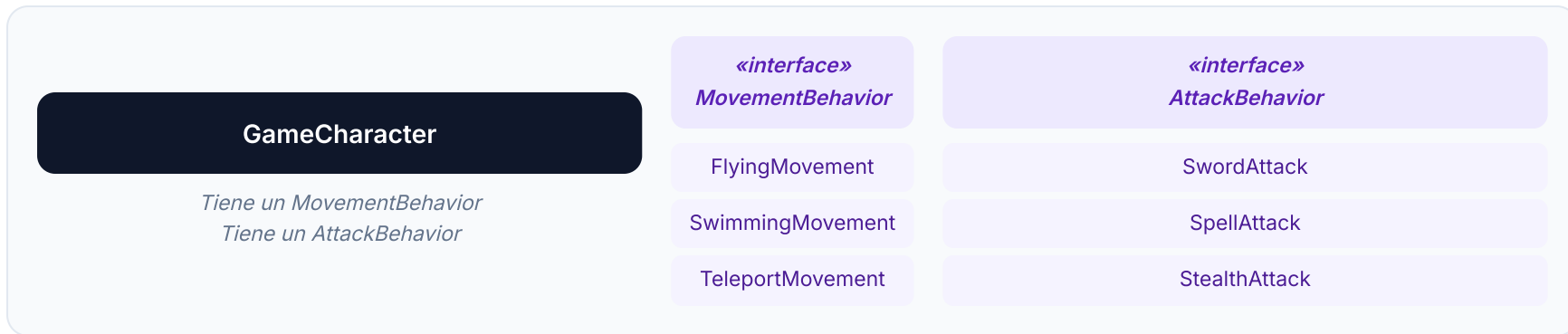
En lugar de *heredar* comportamiento, **componerlo**. El objeto **HAS-A** referencia a objetos que implementan comportamientos, y les delega. Relación IS-A = herencia. Relación HAS-A = composición.

Regla práctica: si te encuentras haciendo `extends` solo para reutilizar código (no para polimorfismo real), cambia a composición. El **Strategy Pattern** es la implementación canónica.

Composición — Cómo funciona el Strategy Pattern

El patrón Strategy en palabras

En lugar de que la clase defina el algoritmo (herencia), la clase **contiene una referencia** a un objeto que encapsula el algoritmo. Ese objeto puede cambiar en tiempo de construcción o incluso en tiempo de ejecución. La clase delega la operación al objeto que tiene compuesto.



 **Intercambiable en runtime:** un guerrero que cae al agua puede cambiar su `MovementBehavior` a `SwimmingMovement` sin que exista una clase `SwimmingWarrior`.

 **Extensible sin explosión:** agregar `TeleportMovement` es una nueva clase de 5 líneas. Con herencia habrías necesitado nuevas subclases para cada combinación de movimiento y tipo de personaje.

IoC, DI & Dep. Inversion

Tres conceptos que todo Java developer debe diferenciar con claridad

Los tres conceptos diferenciados

PRINCIPIO DE DISEÑO

DIP

Dependency Inversion Principle

¿QUÉ DISEÑAR?

Depende de **abstracciones**. Los módulos de alto nivel no deben depender de los de bajo nivel. Ambos dependen de la interfaz.

→ Usa `UserRepository` (interfaz), no `JpaUserRepository`

PATRÓN DE IMPLEMENTACIÓN

DI

Dependency Injection

¿CÓMO HACERLO?

Las dependencias se **inyectan desde afuera** en lugar de crearse internamente. El objeto no hace `new` de sus dependencias.

→ El constructor recibe `UserRepository` `repo` como parámetro

PRINCIPIO ARQUITECTÓNICO

IoC

Inversion of Control

¿QUIÉN CONTROLA?

El **framework controla el flujo**, no tu código. "Don't call us, we'll call you." — Hollywood Principle.

→ Spring crea, configura e inyecta los beans automáticamente

DIP

→ dice qué diseñar →

DI

→ mecanismo de implementación →

IoC

→ el framework orquesta todo

DI — Los 3 tipos de Inyección

✓ Preferida

Inyección por Constructor

```
public AuthService(  
    UserRepository repo,  
    PasswordEncoder encoder) {  
    this.repo =  
    Objects.requireNonNull(repo);  
}
```

Las dependencias son **explícitas e inmutables** (final)

El objeto **nunca puede estar** en estado inválido — sin dependencia, no se construye

Testeable sin framework: solo pasas los mocks en el constructor

⚠ Opcional

Inyección por Setter

```
public void setRepository(  
    UserRepository repo) {  
    this.repo = repo;  
}
```

Útil para **dependencias opcionales** o reconfigurables

El objeto puede estar en estado **parcialmente configurado**

Requiere más cuidado para no romper invariantes

⊘ Evitar

Inyección por Campo (@Autowired)

```
@Autowired  
private UserRepository repo;
```

Conveniente pero **oculta las dependencias**

Requiere framework para funcionar — **no testeable sin Spring**

El objeto puede crearse con campos **null**

IoC & DI — Preguntas de Entrevista EPAM

¿Cuál es la diferencia entre IoC, DI y DIP? Son tres cosas que la gente confunde constantemente.

DIP (Dependency Inversion Principle) es el principio de diseño: depende de abstracciones, no de implementaciones. **DI** (Dependency Injection) es el patrón de implementación: las dependencias se inyectan desde afuera, no se crean con `new` dentro del objeto. **IoC** (Inversion of Control) es el principio arquitectónico: el framework controla el flujo de creación y gestión de objetos, no tu código. La relación es: DIP dice *qué* diseñar, DI dice *cómo* implementarlo, IoC dice *quién* controla. En Spring: el contenedor implementa IoC, usa DI para conectar los beans, y todo el sistema funciona gracias a DIP.

¿Por qué se prefiere la inyección por constructor sobre `@Autowired` en campo?

Tres razones principales. Primera, **inmutabilidad**: con constructor injection los campos pueden ser `final`, garantizando que nunca cambien después de la construcción. Segunda, **testabilidad**: puedes crear el objeto pasando mocks directamente al constructor, sin necesitar Spring en el test. Con `@Autowired` en campo necesitas el contenedor de Spring o reflexión para inyectar los mocks. Tercera, **invariantes claras**: si el objeto no recibe todas sus dependencias, directamente no se puede construir — el fallo es en tiempo de construcción, no en tiempo de uso.

"No me digas qué es el principio. Dime qué problema resuelve, cuándo lo usarías, cuándo no, y dame un ejemplo real."

— Pregunta estándar en entrevistas técnicas EPAM nivel Mid/Senior

OOP	Surgió para resolver el acoplamiento y complejidad de sistemas grandes. Estado + comportamiento.
Encapsulación	El objeto protege sus invariantes. No es getter/setter. Es exponer operaciones, no datos.
Abstracción	Clase abstracta = estado + Template Method. Interfaz = contrato puro. Programar hacia contratos.
Herencia	IS-A. Solo simple en Java. Sealed classes. Fragile Base Class Problem. Usar con criterio.
Polimorfismo	Estático = overloading (compilación). Dinámico = overriding (runtime). El dinámico hace OCP posible.
SOLID	SRP: una razón para cambiar. OCP: extender sin modificar. LSP: subclases sustituibles. ISP: interfaces pequeñas. DIP: depende de abstracciones.
Composición	HAS-A sobre IS-A cuando los comportamientos son combinables o intercambiables. Strategy Pattern.
IoC / DI	DIP = principio. DI = patrón. IoC = arquitectura. Constructor injection es la preferida.